

CE-304T Software Engineering

Lecture - 10

Design and Implementation - II

Design models

- Design models show the objects and object classes and relationships between these entities
- Static models describe the static structure of the system in terms of object classes and relationships
- Dynamic models describe the dynamic interactions between objects

Examples of design models

- Subsystem models that show logical groupings of objects into coherent subsystems
- Sequence models that show the sequence of object interactions
- State machine models that show how individual objects change their state in response to events
- Other models include use-case models, aggregation models, generalisation models, etc.

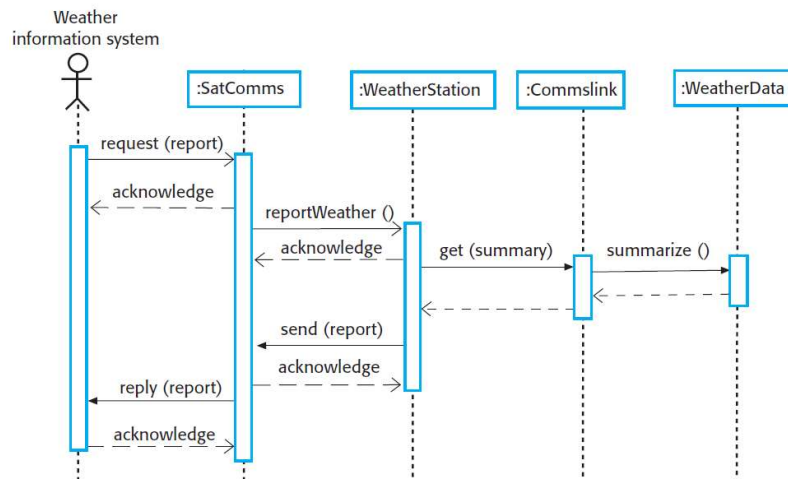
Usman Younis

Sequence models

- Sequence models show the sequence of object interactions that take place
 - Objects are arranged horizontally across the top
 - Time is represented vertically so models are read top to bottom
 - Interactions are represented by labelled arrows, Different styles of arrow represent different types of interaction
 - A thin rectangle in an object lifeline represents the time when the object is the controlling object in the system

Usman Younis

Sequence diagram describing data collection



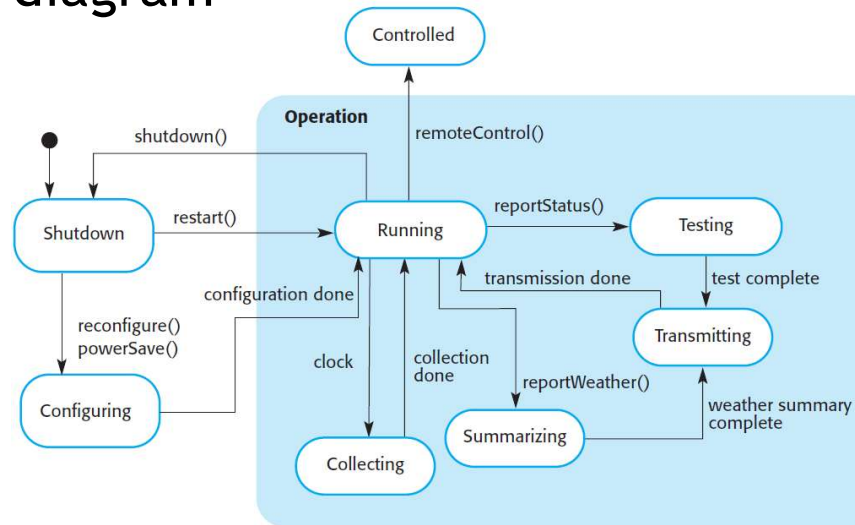
Usman Younis

State diagrams

- State diagrams are used to show how objects respond to different service requests and the state transitions triggered by these requests
- State diagrams are useful high-level models of a system or an object's run-time behavior

Usman Younis

Example: Weather station state diagram



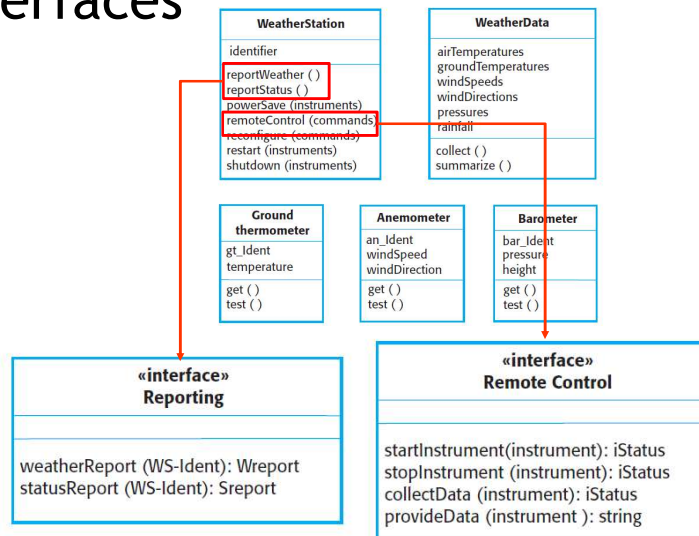
Usman Younis

Interface specification

- Object interfaces have to be specified so that the objects and other components can be designed in parallel
- Designers should avoid designing the interface representation but should hide this in the object itself
- Objects may have several interfaces which are viewpoints on the methods provided
- The UML uses class diagrams for interface specification

Usman Younis

Example: Weather station interfaces



Usman Younis

Key points

- Software design and implementation are interleaved activities. The level of detail in the design depends on the type of system and whether you are using a plan-driven or agile approach
- The process of object-oriented design includes activities to design the system architecture, identify objects in the system, describe the design using different object models and document the component interfaces

Usman Younis

Key points

- A range of different models may be produced during an object-oriented design process. These include static models (class models, generalization models, association models) and dynamic models (sequence models, state machine models)
- Component interfaces must be defined precisely so that other objects can use them. A UML interface stereotype may be used to define interfaces

Usman Younis

Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution
- A pattern is a description of the problem and the essence of its solution
- It should be sufficiently abstract to be reused in different settings
- Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism

Usman Younis

Example: The Observer pattern

Pattern name	Observer
Description	Separates the display of the state of an object from the object itself and allows alternative displays to be provided. When the object state changes, all displays are automatically notified and updated to reflect the change.
Problem description	In many situations, you have to provide multiple displays of state information, such as a graphical display and a tabular display. Not all of these may be known when the information is specified. All alternative presentations should support interaction and, when the state is changed, all displays must be updated. This pattern may be used in all situations where more than one display format for state information is required and where it is not necessary for the object that maintains the state information to know about the specific display formats used.

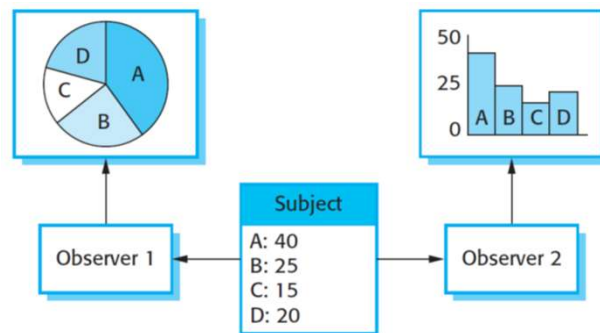
Usman Younis

Example: The Observer pattern

Pattern name	Observer
Solution description	This involves two abstract objects, Subject and Observer, and two concrete objects, ConcreteSubject and ConcreteObject, which inherit the attributes of the related abstract objects. The abstract objects include general operations that are applicable in all situations. The state to be displayed is maintained in ConcreteSubject, which inherits operations from Subject allowing it to add and remove Observers (each observer corresponds to a display) and to issue a notification when the state has changed. The ConcreteObserver maintains a copy of the state of ConcreteSubject and implements the Update() interface of Observer that allows these copies to be kept in step. The ConcreteObserver automatically displays the state and reflects changes whenever the state is updated.

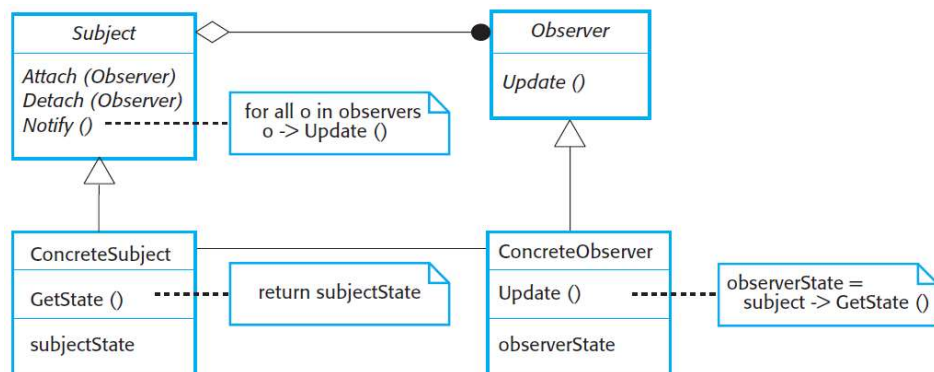
Usman Younis

Multiple displays using the Observer pattern



Usman Younis

A UML model of the Observer pattern



Usman Younis

Design problems

- To use patterns in your design, you need to recognize that any design problem you are facing may have an associated pattern that can be applied
- 1. Tell several objects that the state of some other object has changed (Observer pattern)
- 2. Tidy up the interfaces to a number of related objects that have often been developed incrementally (Façade pattern)

Usman Younis

Design problems

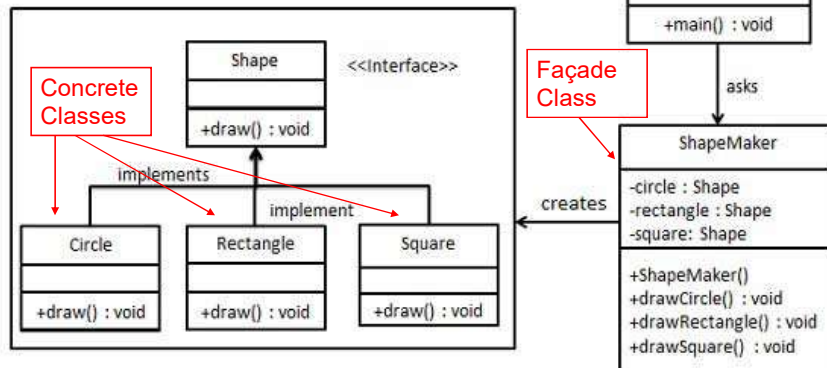
- 3. Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (Iterator pattern)
- 4. Allow for the possibility of extending the functionality of an existing class at run-time (Decorator pattern)

Usman Younis

Facade pattern

https://www.tutorialspoint.com/design_pattern/facade_pattern.htm

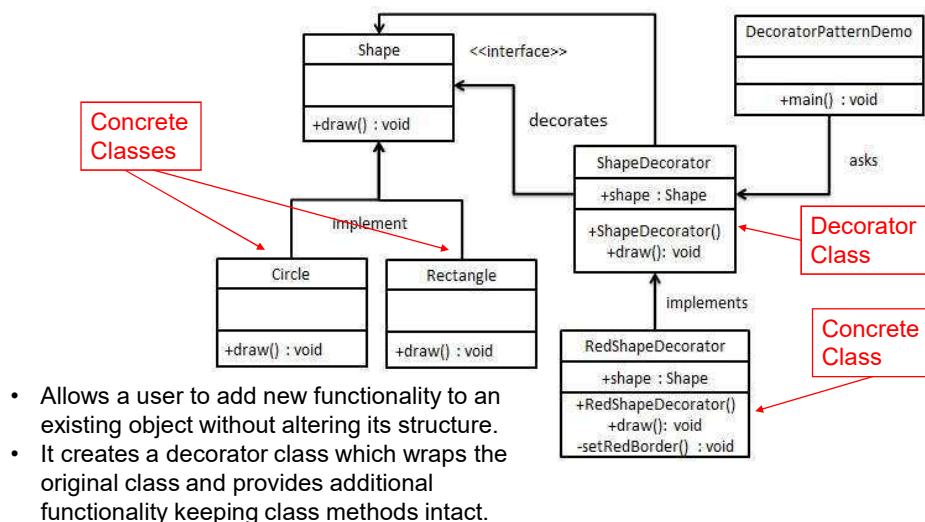
- Facade pattern hides the complexities of the system and provides an interface to the client.
- It involves a single class which provides simplified methods required by client and delegates calls to methods of existing system classes.



Usman Younis

Decorator Pattern

https://www.tutorialspoint.com/design_pattern/decorator_pattern.htm



- Allows a user to add new functionality to an existing object without altering its structure.
- It creates a decorator class which wraps the original class and provides additional functionality keeping class methods intact.

Usman Younis

Implementation issues

- **Reuse** Most modern software is constructed by reusing existing components or systems.
- **Configuration management** During the development process, you have to keep track of the many different versions of each software component in a configuration management system.
- **Host-target development** Production software does not usually execute on the same computer as the software development environment.

Usman Younis

Reuse levels

- **The abstraction level:** You don't reuse software directly but use knowledge of successful abstractions in the design of your software
- **The object level:** You directly reuse objects from a library rather than writing the code yourself
- **The component level:** Components are collections of objects and object classes that you reuse in application systems
- **The system level:** You reuse entire application systems

Usman Younis

Reuse costs

- The costs of the time spent in looking for software to reuse and assessing whether or not it meets your needs
- the costs of buying the reusable software. For large off-the-shelf systems, these costs can be very high
- The costs of adapting and configuring the reusable software components or systems to reflect the requirements of the system that you are developing
- The costs of integrating reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed

Usman Younis

Configuration management

- Configuration management is the name given to the general process of managing a changing software system
- The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system

Usman Younis

Configuration management activities

- **Version management**, where support is provided to keep track of the different versions of software components
- **System integration**, where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components
- **Problem tracking**, where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed

Usman Younis

Host-target development

- Most software is developed on one computer (the host), but runs on a separate machine (the target)
- More generally, we can talk about a development platform and an execution platform
 - A platform is more than just hardware
 - It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment

Usman Younis